Predicting Gene Expression from Chromatin Tracks

ML4G Project 1

Introduction

We predict CAGE gene expression in an unseen cell line (X3, chromosome 1) from chromatin tracks around each gene's transcription start site (TSS), training on cell lines X1 and X2 and scoring with Spearman correlation. Our model is a 1D residual CNN with an MLP head. It reads 11 tracks in 192 bins around the TSS (five chromatin assays, CpG islands and the same assays as long-range context) and regresses $\log(1 + \text{expression})$.

Datasets and tracks

For each cell line we are given a gene table (name, chromosome, TSS, strand), CAGE expression for the training and validation genes, and genome-wide signal tracks (bigWig) for seven assays: DNase-seq and six histone marks. The data is split by chromosome: 14,310 training genes per cell line (chr2–13, 15–18, 20–22), 1,974 validation genes per cell line (chr14, chr19), and 1,984 X3 test genes (chr1). We also use the hg38 reference genome.

For every gene we cut a window around the TSS: 2,048 bp upstream and 1,024 bp downstream, averaged into 16 bp bins (192 bins). Upstream sequence is the more informative side for regulation, and minus-strand genes are flipped so that upstream is always on the left. The input has **11 tracks** $\times$ 192 bins:

- **Five assays** at promoter scale: DNase (open chromatin), H3K4me3 (active promoters), H3K27ac (active promoters and enhancers), H3K4me1 (enhancers), H3K36me3 (transcribed gene bodies).
- **CpG islands**, computed from the genome: the fraction of each bin inside a CpG island (200 bp window, GC $> 50\%$, observed/expected CpG > 0.6). It is identical in all cell lines and marks CpG-island promoters.
- **The same five assays as long-range context**: a window of $\approx$20 kb ($\pm$9,984 bp) around the TSS in 104 bp bins, again 192 bins, to capture distal enhancers.

The repressive marks H3K27me3 and H3K9me3 are not used, as H3K27me3 had 47% of bins with 0 peaks and did not add predictive power in preliminary experiments. Additionally, H3K9me3 is unreliable as its per-gene similarity is almost zero between X1 and X2 (0.05) and weak with X3 (0.29–0.33).

Normalization and transformation

Signal tracks are averaged per bin (missing values set to 0) and transformed with $\log(1 + x)$, because the signal is strongly right-skewed. Each track is then z-scored *within its cell line*, using the mean and standard deviation over all genes and bins of that cell line (training genes for X1/X2, the test genes themselves for X3; no labels are involved). This removes differences in overall signal scale between cell lines, which matters for transferring to the unseen X3. The CpG track is already a 0–1 fraction and is left unchanged. The target is $\log(1 + \text{expression})$.

Model architecture

A 1D residual CNN followed by an MLP, with 7.05M parameters. A stem convolution (kernel 5, 11$\to$32 channels, BatchNorm, GELU) feeds three residual blocks with 64, 128 and 256 channels. Each block has two kernel-5 convolutions with BatchNorm, GELU and a skip connection (a 1$\times$1 convolution when the shape changes), and no dropout. The strides are $(1, 2, 2)$, so the length goes $192 \to 192 \to 96 \to 48$. The 256×48 output is *flattened* rather than pooled, so the head keeps the position of each feature relative to the TSS. The head is $12{,}288 \to 512 \to 128 \to 1$ with BatchNorm, GELU and dropout 0.5 after each hidden layer. Training uses AdamW (learning rate 10^{-3}, weight decay 0.05), a cosine schedule over 20 epochs, batch size 128 and a Huber loss ($\delta = 1$) on the $\log(1 + x)$ target.

Results

	Spearman (val)	Spearman (test, X3 chr1)	Train loss	Val loss
Current model	≈ 0.82	0.793	≈ 0.34	≈ 0.41

Validation Spearman reaches about 0.82 within the first few epochs and then stays roughly flat while the training loss keeps falling. The gap between the training loss (0.34) and the validation loss (0.41) points to mild overfitting, which the dropout and weight decay only partly control. The test score of 0.793 is a little below the validation score, as expected for an unseen cell line.

Contributions

Tasks weren't explicitly assigned, but the work was roughly split as follows: Dea K. ran the initial data exploration and preprocessing, along with testing a simple linear model, that passed the baseline. She also tested different model architectures and hyperparameters, and implemented the final model. On the other hand, Remi T. implemented the model and training pipeline, and ran the final training and evaluation. He also wrote the report. While doing so, both of us discussed the results and the next steps, and we helped each other with debugging and code review.